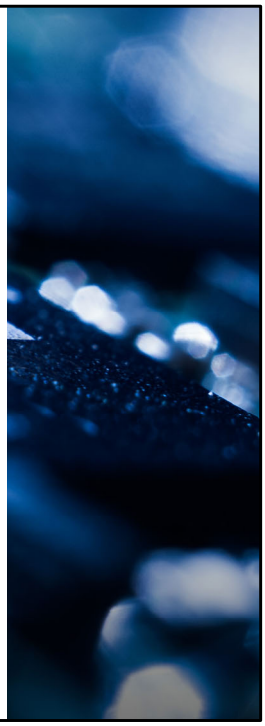
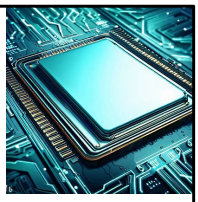
SC1006
Computer Organisation and Architecture
Cache

Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg



- This chapter is on Cache Memory Management. This is a module in the processor that is able to improve the overall system performance significantly based on the principle of locality exhibited by most application programs. More details in this video.

Cache



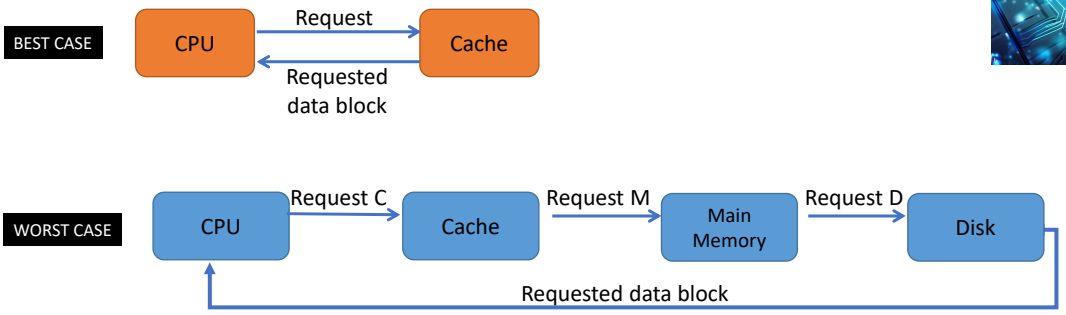
- Issue: **CPU** speed is typically much **faster** than **external memory**.
  - CPU speed in Ghz region
  - External Memory Speed in 100s of Mhz region.
- **Need a fast memory** to act as a **buffer** between the Main memory and CPU.
- The purpose of cache memory is to speed up accesses by storing/fetching **recently used data** closer to the CPU instead of the main memory (slower access).
- Potentially able to **improve the overall system performance** drastically.

Now, CPU typically runs at a higher speed compared to the external memory.
So the external memory becomes the bottleneck and slows down the system performance.

To increase the overall system performance, we need a fast memory to act as a buffer between the CPU and External Memory.
And that piece of fast memory is known as Cache.

We'll see later that a well-designed cache could potentially improve the system performance significantly.

CPU Memory Access



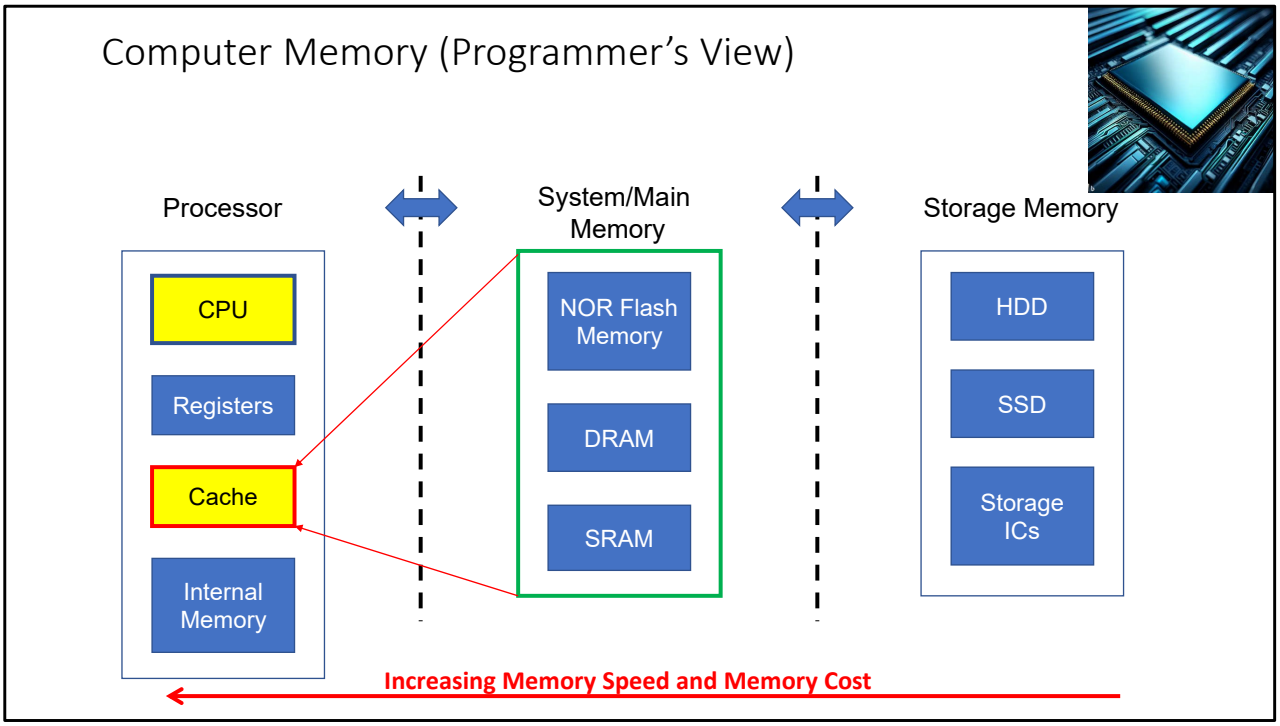
- To access a particular piece of data, the CPU first sends a request to its nearest memory, usually cache.
- If the data is not in cache, a query is then sent to the main memory.
- If the data is not in main memory, then the request goes to disk.
- Once the data is located, the **required data and a number of its nearby data elements** are fetched into cache memory simultaneously.

This slide shows you what really happen between the CPU, cache and main memory.

In a processor with cache, the CPU will first visit the cache to see if it has the data the CPU needs.
If the data is there, then the requested data is transferred and the processor proceeds with other operations.
If data is not in the cache, then CPU will need to look in the main memory or the HDD for the data.

Once data is found, it'll be fetched into the cache, together with some other nearby data elements.

How the main memory data is stored in the cache depends on the mapping scheme used. And this is what we'll be going through in the next a few slides.



Let's re-visit a previous slide on computer memory from the perspective of a programmer.

A program's code and data are stored entirely in the Storage memory.

The specific sections of code and data are transferred to the System memory during runtime.

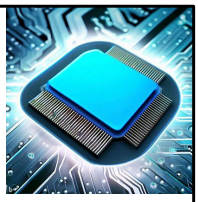
A subset of the code and data which is recently or frequently used is stored in the cache for fast access by the CPU.
Different subset of code/data will be loaded to the cache as the program executes
The objective is to have the subset that CPU need most at that instance to minimise the time need to retrieve these information.

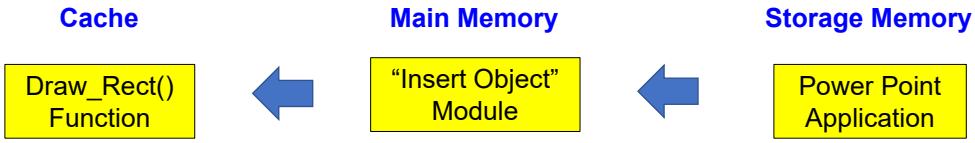
Just to recap, CPU will first visit the cache to look for the information it needs, it will proceed to the System memory if it can't find the required information, and further check out the storage memory if it can't find the information in the System memory.

You will see the system memory being referenced by the name "main memory" in the

rest of chapter. In this course, system memory and main memory are referring to the same piece of memory.

Cache Design - Introduction





- Example: Power Point Application
- Entire Application is stored in Storage Memory
- The user is trying to insert some object into the powerpoint slide
  - The corresponding code for object insert is transferred to Main memory for execution
- At the instance, the user is drawing a rectangle
  - The corresponding code for the Draw_Rect() function finds its way into the cache eventually.

This slide gives you an illustration of what happens in a Processor with cache when running a power point application.

The entire powerpoint application is too large to be stored in the main memory so it is stored in the storage memory instead.

User will only be using one of the many features so only those needed is loaded to the main memory.
In this case, it's the insert object module that is loaded.

Even within the insert object module, users will only be using a sub-set of what the module provides.
In the case, the user is trying to draw a rectangle.
The particular function that he needs will eventually find its way into the cache since it is the most recently and frequently used group of code.

Principle of Locality

- Now that we know cache only needs to contain a subset of the main memory to work
- In what way should the data be transferred between main memory and cache in order to maximise the efficiency of a cache?
- Efficiency meaning how much improvement can the cache brings to the system. This is related to the probability the CPU is able to find the information that it needs in the cache.
- For that, we need to understand the attributes of how information is organised and accessed when a program is executed.
- This is summarised in the concept called Principle of Locality

From previous slides, we know that CPU only needs a subset of the entire program during runtime.

This is why cache size can be very small compared to system and storage memory.

So the next question we need to answer is what data should be stored in the cache in order to maximise the resultant system performance?

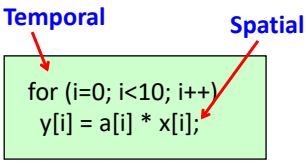
To maximise the system performance, we need to maximise the cache efficiency.

The efficiency here is directly related to the probability of CPU finding what it needs in the cache.

To enable that, we need to understand the attributes of how code and data are organised and accessed when a program is executed.

This is summarised in the concept called principle of locality.

Principle of Locality



- The **principle of locality** tells us that once a byte in a program is accessed, it is likely a **nearby data element** will be needed soon.
- There are two principles of locality governing this behaviour
- Locality of **Space** (or Spatial Locality)
  - **Code/Data that is nearby each other** is likely to be accessed together
  - Transfer of data between main memory and cache is done in blocks to leverage on this behaviour
- Locality of **Time** (or Temporal Locality)
  - **Recently accessed code/data** is more likely to be accessed again
  - Used to decide which item to replace in the Cache

When a byte of code or data is accessed within a program, it is very likely that its nearby code or data will be needed soon.
This attribute is known as the principle of locality

There are two types of locality
Locality of space and Locality of time
Also known as Spatial and Temporal Locality

Spatial locality refers to the scenario where the code/data that are located near to each other are likely to be accessed together
That leads to the cache design where if a particular byte required by the CPU is not in the cache, then the cache module will transfer that byte together with its neighbours to the cache. That is, any transfer between the cache and main memory always happen in blocks and not a single byte. This is so that subsequent access will result in cache hit. Cache hit means that CPU is able to find the required information in the cache.

Temporal locality, on the other hand, refers to the scenario where the recently accessed code/data is likely to be accessed again.

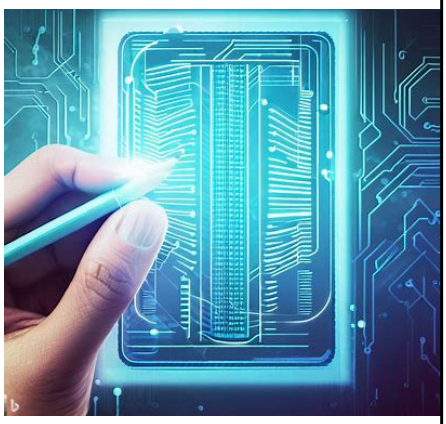
This is a sort of justification on why a cache will be effective even though its size is small.
It is also one of the considerations used when designing the cache replacement policy

The small “for loop” shown here illustrate the two principles.
The for loop had 10 iterations, with each iteration performing a sum of product between array a and x, and storing the result to array y.
The sequential access of the arrays illustrate the concept of spatial locality.

> When one element of an array is used, the chances of subsequent elements being accessed is high.
> The 10 iterations implies the code for the sum of product will be executed 10 times, an illustration of temporal locality in operation.

Cache Mapping Scheme

- We understand now that data transfer between main memory and cache is done in blocks instead of individual bytes to leverage on the principle of locality.
- Another attribute of cache design that affects the efficiency of the cache is the cache mapping scheme.
- Cache mapping scheme deals with how each main memory block is mapped to the a particular cache block, e.g. Main memory block #0x80 is mapped to cache block #0 (index 0).
- There are three basic cache mapping schemes
  - Direct Mapped
  - Set Associative
  - Fully Associative
- We will only touch on Direct Mapped Cache in this course, rest of the mapping scheme will be discussed in Advanced Computer Architecture Course.

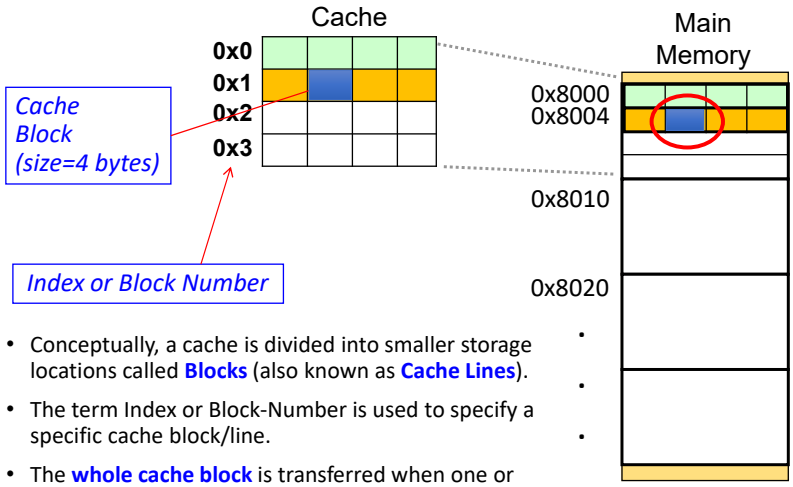


Another attribute of the cache design that will affect the efficiency is the cache mapping scheme
This deals with how each block in the main memory is mapped to the cache

There are three basic types of mapping scheme
Direct Mapped
Set Associative and
Fully Associative

For this course, we will only touch on the Direct Mapped Cache. The other two mapping scheme will be discussed in the Advanced Comp Arch module.

Terms used in Cache Mapping



- Conceptually, a cache is divided into smaller storage locations called **Blocks** (also known as **Cache Lines**).
- The term Index or Block-Number is used to specify a specific cache block/line.
- The **whole cache block** is transferred when one or more bytes in the block is needed.

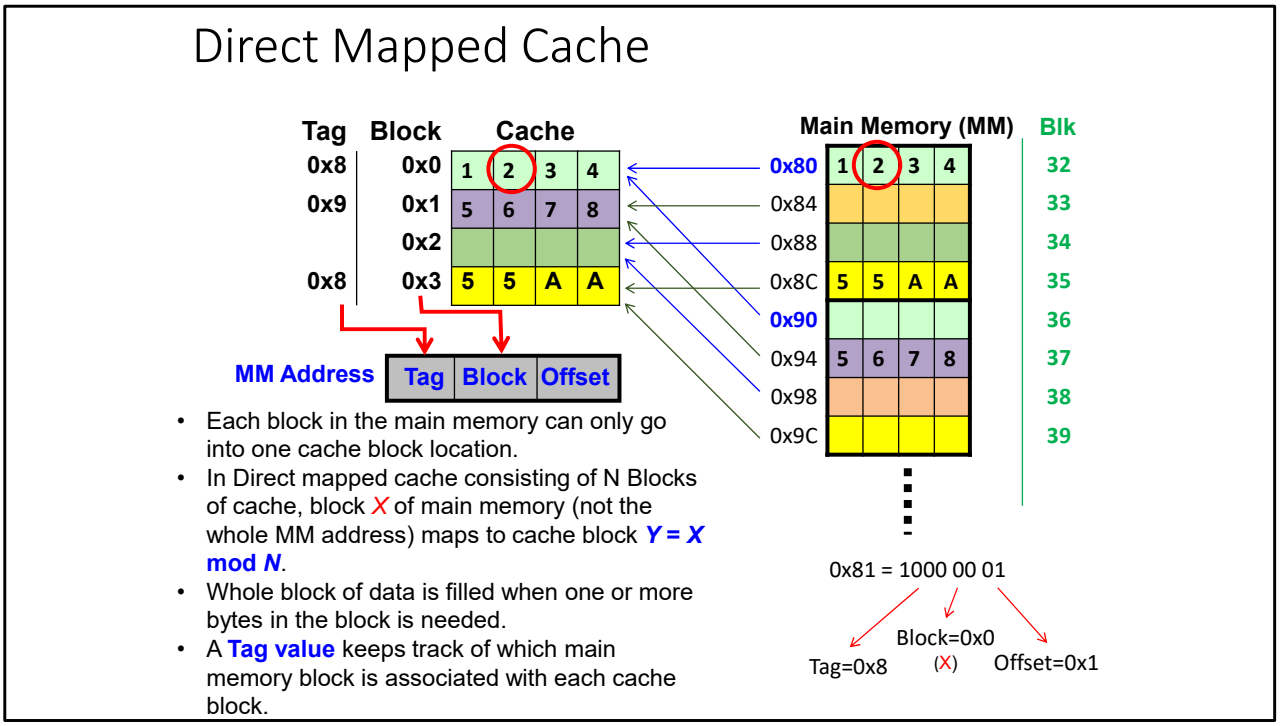
Before we go into the details of cache mapping scheme, let's go through some basic terms that we will be using.

First is the Cache Block (or Cache Lines).
Cache is divided into blocks consisting of multiple bytes
For this example, the cache block contains 4 bytes each.

Each Cache block in the cache is numbered. This is known the cache block index or block number.
First cache block of the cache has a index 0.

In cache analysis, the main memory is typically organised in the same format as the cache, i.e if the cache has a block size of 4 bytes, the main memory will also be arranged in blocks of 4 bytes each.

When a particular byte in the main memory is needed, the entire block is transferred to the cache instead of the particular byte.
This as mentioned earlier, is to address the principle of locality.



For Direct mapped cache, each main memory block can only go into one specific block location in the cache.

The way they are allocated is given by the formula X mod N, where X is the main memory block number and N is the total number of cache blocks available.

So MM block 0 goes to cache block 0, MM block 1 goes to cache block 1 and so on. MM Block 4 for this cache structure will wrap around and go to cache block 0. In this example, MM BLK32 goes to Cache BLK 0, the mapping continue sequentially until MM BLK36 where it wraps around and get mapped to Cache BLK 0.

As mentioned in the previous slides, although only one data byte is needed, the whole block of data is copied to the cache.

Question here would be: How do we know which main memory block does the data in the cache correspond to?
For that, we need to look at the tag value, which is used to uniquely identify which MM block is in the cache.

For example, block starting 0x80 will be place in cache block 0 and have a tag value of 0x8, 0x8C will be placed in block 3 and have tag value of 0x8 as well. 0x94, on the other hand, will be placed in block 1 and has a tag value 0x9.

By now, you must be wondering how I derive these values.

Take for example 0x81.

The first step is to partition the MM address into three fields consisting of TAG, BLK and OFFSET.

To know how many bits the 'offset' field supposed to have, we need to understand the definition of the offset field.

An offset specifies the position of the target byte from the start of the cache block.

For this example, the cache block size is 4, the offset field will consist of 2 bits as that is the number of bits needed to support 4 different address representation. For ease of calculation, you just need to use LOG2 of 4.

Next is the 'BLK' field. Similarly, this field describes which block the target data is transferred to, so if the cache contains 4 block, 2 bits is required for the 'BLK' field.

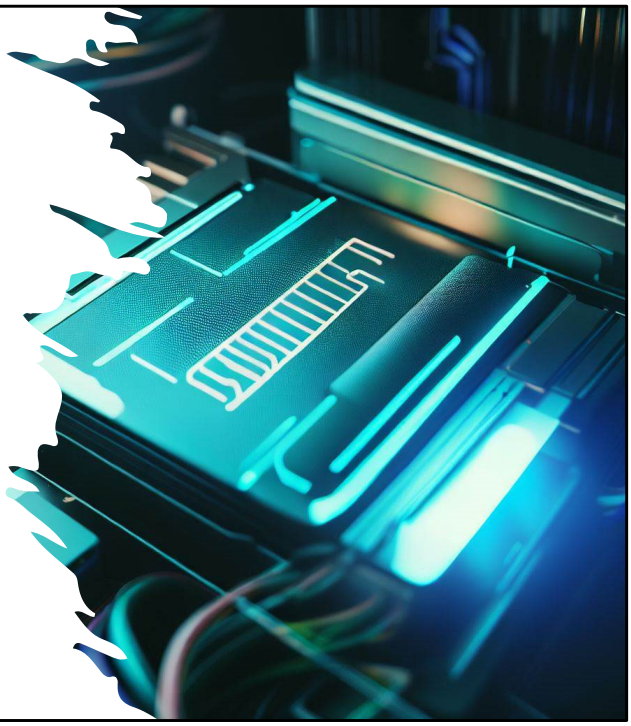
Lastly, the TAG field, which is used to uniquely identify each MM block, is derived from subtracting the bits allocated to offset and BLK. From this case, MM address range is 8 bits, so number of TAG bits = 8-2-2 = 4.

After we partition the MM address according to the TAG, BLK and OFFSET field, we can easily determine which cache block this particular MM block will be transferred to by looking at the BLK field.

In this example, we can see that the MM block which contains 0x81 will be mapped to Cache Block 0 and the corresponding TAG value is 0x8.
We can also tell that the actual byte of interest is one byte away from the start of the cache block.

Cache Mapping (Elaboration on basic principle)

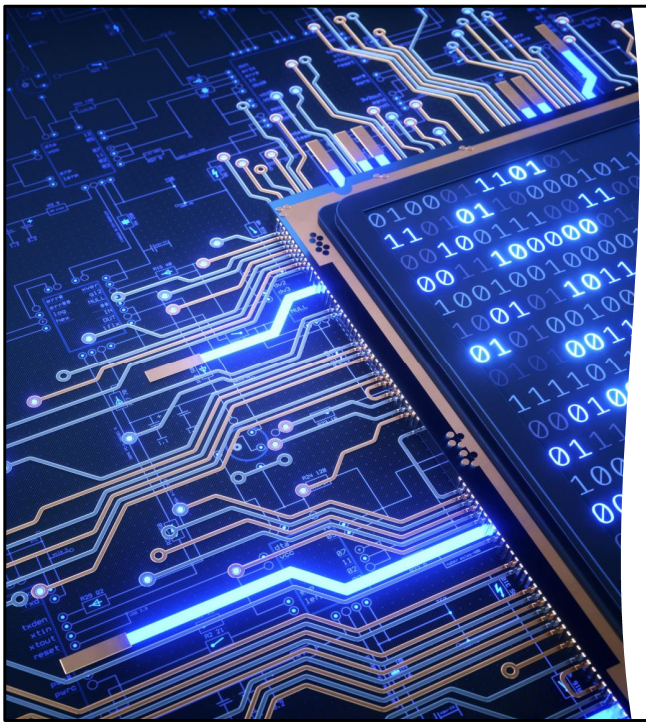
- Cache mapping
  - Allocates the data in the entire main memory into the cache which is of a much smaller size.
  - Able to uniquely identify each and every main memory location within the cache via the cache way of addressing: Block Index, Offset of the data within the block, and the corresponding Tag Value of the block.
- Since the CPU will use the MM address as the reference to retrieve the information, it is only natural that the MM address is used as the basis to enable the mapping process.



This slides elaborate on what we have learn so far regarding cache mapping scheme.

Cache mapping involves the following tasks
Design a systematic way to map the main memory to a cache whose size is smaller than the main memory.
Allow the CPU to retrieve the information (code or data) it needs from the cache, based on the MM address of the target information.

Since the CPU will use the MM address as the reference to retrieve the information, it is only natural that the MM address is used as the basis to enable the mapping process.



Cache Mapping (Elaboration on basic principle)

- The **number of bits allocated to each field** is a result of the structure of the cache.
  - **Offset** refers to the targeted data's location offset from the start of the cache block. Something like an address within the cache block. So if the size of a cache block is 16, one would need 4 bits in the OFFSET field to address these 16 locations.
  - **Block** refers to the index of the cache block that the targeted data will be mapped to. If there are 8 cache blocks for example, then one would need 3 bits in the BLOCK field to fully represent the cache block index.
  - **Tag** bits are the left over of target data's main memory address after partitioning for Offset and Block Index. Having this information will allow the cache system to **uniquely identify** the data in the cache.

- This slide contain detail description of each of the TAG, BLK and OFFSET field.
- The content has been discussed in previous slide so I will not elaborate further.
- You can pause the video here to review through the points again.

Next, let's go through a work example to check our understanding of the concepts we have learnt so far for direct mapped cache mapping scheme.

Given a system with
MM size 64KBytes,
Cache Size 256 Bytes
Cache Block Size 16Bytes
Where would the Data at MM address 0x1106 be mapped to?

Using the concept we learnt on the TAG:BLK:OFFSET fields for direct mapped cache, Starting with the offset first, cache block size = 16 implies offset field is LOG2(16) = 4 bits.
Number of cache blocks is not given but that can be derived by dividing the cache size by the cache block size.

So there are 16 cache blocks in the cache. That implies BLK field is also 4 bits. The TAG field is obtained by subtracting the MM address bit (16 bits for 64KBytes) with OFFSET and BLK bits.

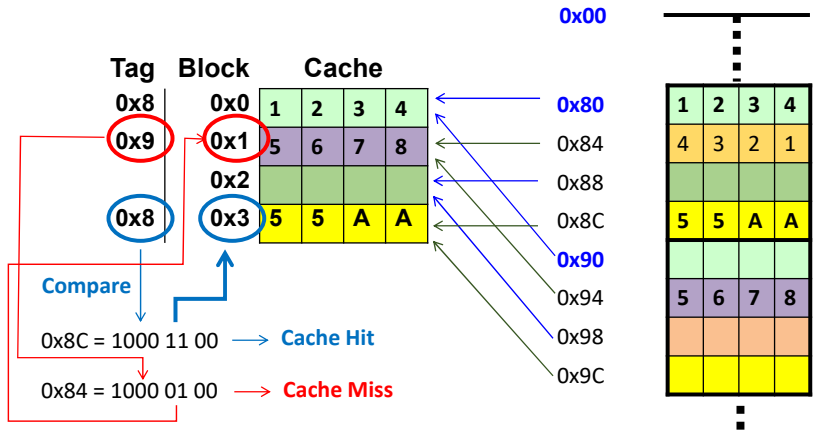
So number of TAG bits = 8.

That means that the Mapping Format is 8:4:4

When doing the cache mapping analysis, always remember to first convert the MM address to binary; performing analysis with hexadecimal MM address will very likely result in careless mistakes.
After converting 0x1106 to binary and applying the mapping format
We can see that the data at 0x1106 will be mapped to cache BLK 0 and has a TAG value of 0x11.

The target byte is located at byte 6 from the start of cache block.
Byte 0 is the first byte of the cache block.

Data Retrieval Example (Direct-Mapped Cache)



Total Memory = 0xFF = 256 bytes => 8 address bits.
Cache Size = 16Bytes
Cache Block Size = 4 Bytes => 2 address bits for offset.
# of Blocks in Cache = 16/4 = 4 Blocks => 2 address bits (Block index)
# of TAG bits = 8-2-2 = 4 bits.

After the work example in the previous slide, let's go through the data retrieval process to see how CPU evaluates whether a particular access is a cache hit or cache miss.

Using the procedure mentioned in earlier slides, we can derive the mapping format for this cache structure
4:2:2
Note that mapping format will change if the cache structure change e.g. change in cache size, cache block size and/or MM address range.

Supposed after some data exchange, CPU wants to retrieve data stored at location 0x8C.
Applying the 4:2:2 format on the 0x8C, BLK value is '3' so CPU proceed to check out cache block 3.
To find out whether the data stored at cache block 3 comes MM block that contains 0x8C's data.

It compares the TAG value of cache block 3 and the TAG value derived from 0x8C MM address.
Result is a match, that implies the data in cache block 3 does indeed comes

from MM. This is known as a Cache Hit.

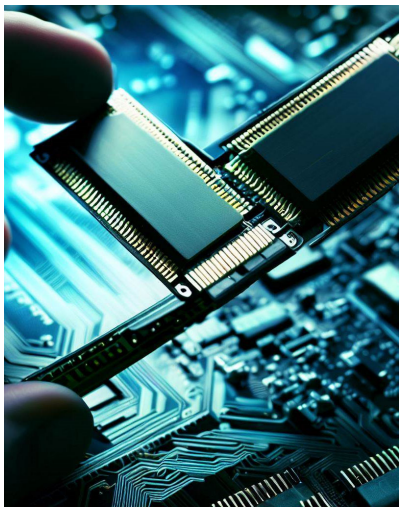
The CPU then proceeds to access 0x84.
Applying the same procedure again, we realise that this time, the TAG value of cache block 1 is different from the TAG value derived from the 0x84 MM address. This means that the data in cache block 1 does not come from MM block where 0x84's data is resided. This is known as a Cache Miss.

If there is a Cache Hit, CPU will retrieve the data and proceed with other operations.

If there is a Cache Miss, the corresponding MM block will be transferred into the cache.

Cache Memory Replacement Policy

- The direct mapped cache doesn't really need a cache replacement policy as its mapping is one-to-one.
- When there is a need to transfer a block of data to the cache but the cache is fully occupied, there is a need to decide which cache block to evict/purge in order to free up space for the new data.
- The algorithm used to decide which block gets evicted is designed based on some Cache Replacement Policy. Two examples illustrated below
- **Least recently used (LRU)** algorithm keeps track of the last time that a block was assessed and evicts the block that has been unused for the longest period of time. The disadvantage of this approach is its complexity: LRU has to maintain an access history for each block, which ultimately slows down the cache.
- **First-in, first-out (FIFO)** is a popular cache replacement policy. In FIFO, the block that has been in the cache the longest, regardless of when it was last used.



The other part of cache storage design deals with replacement policy.

While the direct mapped cache doesn't really need a cache replacement policy as its mapping is one-to-one, its good to understand the general concept behind replacement policy.

Set associate or fully associative cache, which you will cover in Advanced Comp Arch module, will need cache replacement policy to decide which blocks get replaced when the cache is full.

In general, if the cache is full and a new block needs to be introduced into the cache, then the cache module will need to decide which cache block to evict to make space for the new block.

For Direct Mapped Cache, since the mapping is one-to-one, the block to evict is known before hand so there isn't a need to have any algorithm to manage the block replacement.

For Set Associate and Fully Associate Cache, which is not covered in this course, there

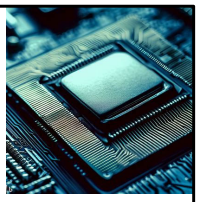
is an option to choose from two or more blocks to evict. Hence, a replacement policy has to be put in place.

Two of these methods are described here

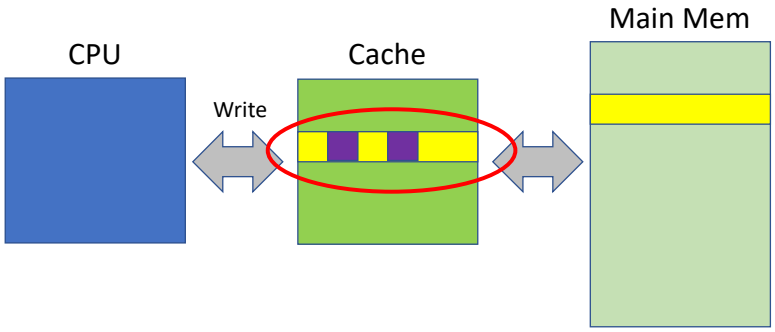
Least Recently Used (LRU) algor. This scheme will evict the cache block that has not been used for the longest time. It's a very efficient scheme since the algor conform well to the locality principles. But implementation is more complex and costly as the cache needs to keep an access history for each block.

FIFO. Another popular scheme with simpler implementation. It basically replace the first block that comes into the cache. It conforms approximately to the locality principles, just that it doesn't take into account of the scenario where a cache block is used multiple times. This typically result[s in lower efficiency compared to LRU.

Cache Write Policy



- Locations in cache that are written into by CPU is known as **Dirty Block**
- Cache replacement must take into account **dirty blocks**, those blocks that have been updated while they were in the cache.
- Dirty blocks must be written back to memory. A **write policy** determines how this will be done.
- There are two types of write policies: **write through** and **write back**.



So far, our discussion has always been related to CPU read operation.
In the next few slides, we will be looking at effects on cache when CPU perform write operation.

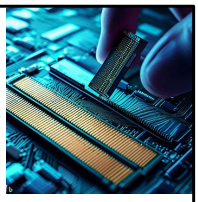
A special marking is done to the cache block which the CPU writes into and these blocks are known as Dirty Blocks.

Cache replacement needs to take into account of dirty blocks.

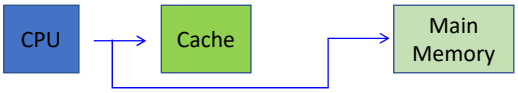
E.g one main memory block was cached. And CPU write into some location in this block within the cache.
The content of the block in cache and main memory is now different.
Meaning that if we want to evict this block, we'll need to first write back this block to the main memory to maintain data coherence.

There are two types of write policies: write through and write back.

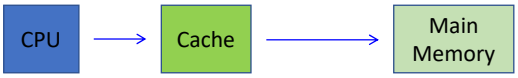
Cache Write Policy (**Write Hit**)



- **Write through** updates cache and main memory **simultaneously** on every write.



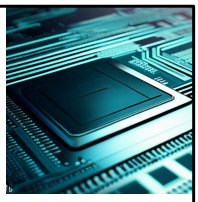
- **Write back** (also called copyback) updates memory **only** when the cache block is selected for replacement.



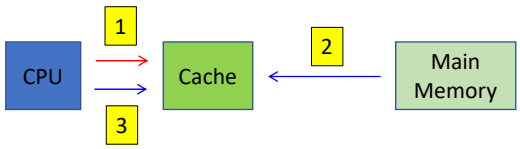
Write through means that if CPU writes to a cache location, it'll also update the main memory at the same time.
This scheme prevents cache coherency issue but increases the traffic on the system bus.

But for a Write back scheme, CPU will only write to the cache, and the main memory will be updated only when the cache block Is evicted.
This scheme doesn't overload the system bus but may potentially have cache coherency issue since the copy of the data in the cache and main memory may be different.
The cache module will also have to keep track of the status of the cache block data to see if a write back to the main memory is required when a cache block is evicted.

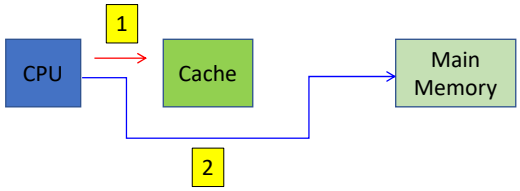
Cache Write Policy for **Write Miss**: CPU wants to write to a certain main memory location but found that it is not in the cache



- **Write allocate** => fetch on write. A write miss will cause the data block at the write-miss address to be **loaded to the cache**.



- **Write-no-allocate** => A write miss will not cause the data block to be loaded to the cache. Data **Write** is done **directly** to its location in **main memory**.



Note that the previous slide applies to the Write Hit scenario.

Correspondingly, there are two ways in which CPU can handle write miss.
Write miss here refers to cases where CPU want to write to a certain main memory location but found that it is not in the cache.
A cache miss could be either a read miss or a write miss.

The two ways in which CPU handles write miss are
Write allocate.

Under this scheme, the corresponding MM block will be loaded to the Cache when there is a write miss.
CPU will follow with a read to the cache to retrieve the data
Doing so allows future read/writes to the same block to yield a Cache Hit.
Follow one of the two procedures for **Dirty Blocks.**

Write no-allocate

With this scheme, CPU will write directly to the corresponding MM block if there is a Write Miss.
Doing so simplifies the write operation but didn't update the cache for future operation.

Which policy to use depends on the nature of the program code and data.

Cache Performance-Related Terminologies

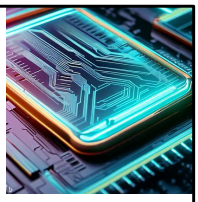
- **Cache Hit**
  - Data is found in the cache; either a read hit or a write hit
- **Cache Miss**
  - Data is not found in the cache; either a read miss or a write miss
- **Cache Hit rate (H)**
  - Percentage of time data found in the cache
- **Cache Miss rate**
  - Percentage of time data not found in cache.
    Miss rate = 1 - Hit rate = 1 – H

Some definition of the key parameters used for calculating Effective Access Time.

We have encountered most of them previously, so I won't go through them again.

You can pause the video briefly to review the definitions.

Effective Access Time



Sequential Access of Cache and Main Memory, i.e. access do not overlap.

- The performance of hierarchical memory is measured by its effective access time (EAT).
- EAT is a weighted average that takes into account the hit ratio and relative access times of successive levels of memory.
- The EAT for a two-level memory is given (as a general form):

$$EAT = H \times Access_C + (1-H) \times \textit{Miss Penalty}$$

- $Access_C$ = Access times for cache
- Miss Penalty = Time need to access the data when there is Cache Miss
- If we assume that data access to Cache and Main memory do not overlap, then Miss Penalty = $Access_C + Access_{MM}$, where $Access_{MM}$ is the access time for main memory

$$EAT = H \times Access_C + (1-H) \times (Access_C + Access_{MM})$$

applies only to READ operations as write operations will be more complicated (with combination of write-through/back, write-allocate/no-allocate policies)

Effective Access Time, EAT in short, is the weighted average of the access time during cache hit and cache miss.

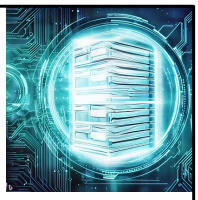
Now, CPU always visit the cache first when it needs information.
If it can find the information in the cache, it's a cache hit and the access time incurred will be the Cache access time.
If CPU cannot find the information in the cache, it's a cache miss and CPU will need to visit the Main Memory to retrieve the information.

If we further assume that access to cache and main memory happen sequentially and do not overlap,
Then access time during cache miss will be cache access time + main memory access time.
The cache access time is incurred when CPU first visits the cache to look for information.
The main memory access time is incurred when CPU realised that it is a cache miss and need an **additional** MM access time to retrieve the data.

As mentioned, the effective access time is a weighted average of the two access

timing and is calculated by applying the probability of each of the scenarios to the two access time components. Resulting in the expression you see in the slide.

Effective Access Time (Example)



Sequential Access of Cache and Main Memory

- Consider a system with a **main memory access time of 200ns** supported by a **cache** having a **10ns access time** and a **hit rate of 99%.**
- If the **accesses do not overlap**,

  The EAT is:

  0.99(10ns) + 0.01(10ns + 200ns) = 9.9ns + 2.1ns = 12ns.
- This equation for determining the effective access time can be extended to any number of memory levels.

Applying what we learn about calculation of the EAT in one work example.
System has a MM access time of 200ns, cache access time of 10ns and cache hit rate of 99%
Remember the assumption that access to memories do not overlap.
EAT is obtained by using the expression in the previous slide for READ opertaion.
We will obtain 12ns.
Here is a good example showing how a small cache can realise a significant improvement in Effective Access Time of the memory system.
Note that it is common to have cache hit rate of higher than 90%.
With that, we come to the end of the Cache Memory System module. Next lecture will be on Virtual memory management.